2026-4-27 C++类和对象：下

- 1. 再探构造函数
  - 1.1初始化列表
- 2.类型转换
- 3.static成员
- 4.友元
  - 4.1什么是友元？友元的分类是什么？
  - 4.2 友元的作用是什么？怎么使用友元？
  - 4.3 友元的特性是什么？
  - 4.4 友元存在的意义是什么？
- 5. 内部类
- 6. 匿名对象
- 7. 对象拷贝时候的编译器优化

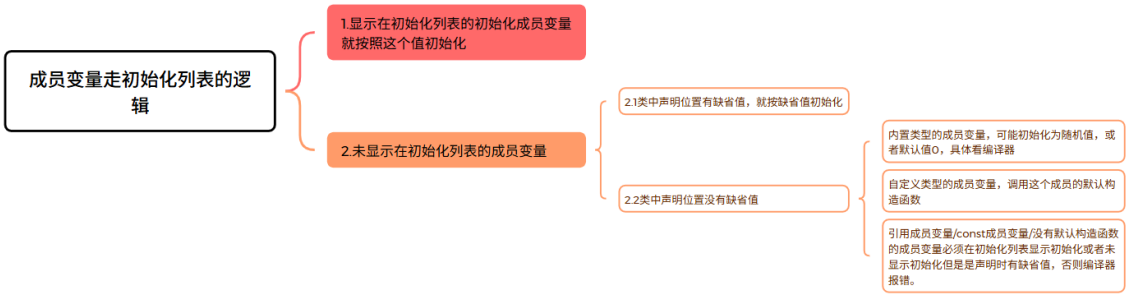
1. 再探构造函数

之前我们实现构造函数的时候，初始化成员变量主要使用函数体内赋值的方式。实际上，构造函数初始化还有一种方式就是--初始化列表。初始化列表的使用方式是以一个冒号开始的，接着是以一个逗号分隔的数据或成员列表，每一个“成员变量”后面跟一个放在括号中的初始值或者表达式。

```
#include<iostream>
using namespace std;
class Time
{
public:
    Time(int hour)
        :_hour(hour) {//用初始化列表初始化
        cout << "Time()" << endl;
    }
private:
    int _hour;
};
int main() {
    int i = 0;
    Time t1(i);
    return 0;
}
```

1.1初始化列表

1. 每个成员函数在初始化列表的时候只能出现一次，语法上理解初始化列表可以认为是每个成员变量定义初始化的地方。
2. 引用成员变量，const成员变量，**没有默认构造的类类型变量**，必须放在初始化列表位置进行初始化，否则会编译报错。
3. C++11支持在成员变量声明的位置给缺省值，这个缺省值主要是给没有显示在初始化列表的成员使用的。
4. 尽量使用初始化列表初始化，因为那些不在初始化列表里的成员变量也要走初始化列表，如果该成员在声明的时候给定一个缺省值，那编译器则会采用初始值。如果没有缺省值，对于那些没有显示在初始化列表的初始化内置类型成员是否初始化取决于编译器，C++并没有明确的规定。对于没有显示在初始化列表初始化自定义类型成员会调用这个成员类型的默认构造函数。但是如果这个成员没有默认的成员函数，则编译器会报错。
5. 初始化列表中按照成员变量在类中声明的顺序进行初始化，跟成员在初始化列表中出现的顺序无关。但是建议声明和初始化列表的顺序保持一致。



示例2：

```
class Time
{
public:
    Time(int hour)//这个不是默认构造
        :_hour(hour) {
        cout << "Time()" << endl;
    }
private:
    int _hour;
};
class Date {
public:
    Date(int& x, int year = 1, int month = 1, int day = 1)
        :_year(year)
        , _month(month)
        , _day(day)
```

```
        , _t(12)
        ,_ref(x)
        ,_n(1)
    {
        // _t 没有默认构造，已在初始化列表中显式调用 Time(int)
        // _ref 是引用，必须在初始化列表中初始化
        // _n 是 const，必须在初始化列表中初始化
}
    }
    void Print() const
    {
        cout << _year << "-" << _month << "-" << _day << endl;
    }
private:
    int _year;
    int _month;
    int _day;

    Time _t;//没有默认构造
    int& _ref;//引用
    const int _n;//const
};
int main() {
    int i = 0;
    Time t1(i);
    return 0;
}
```

示例2：

```
class Time
{
public:
    Time(int hour)//这个不是默认构造
        :_hour(hour) {
        cout << "Time()" << endl;
    }
private:
    int _hour;
};
class Date {
public:
    Date( )
        :_month(2)
    {
```

```
        cout << "Date()" << endl;
    }
    void Print() const
    {
        cout << _year << "-" << _month << "-" << _day << endl;
    }
private:
    int _year=1;
    int _month=1;
    int _day;

    Time _t=1;//没有默认构造
    const int _n = 1;
    int* _ptr=(int*)malloc(12);//const
};
int main() {
    Date d1;
    d1.Print();
    return 0;
}
```

下面程序的运行结果是什么？

```
class A {
public:
    A(int a)
        :_a1(a)      // 注意这里初始化的是 _a1
        ,_a2(_a1)    // 这里用 _a1 初始化 _a2
    {}
private:
    int _a1 = 2;  // 声明顺序：_a1 在前
    int _a2 = 2;  // _a2 在后
};

int main() {
    A aa(1);
    aa.Print();
}
// 运行结果：1 1
// 因为 _a1 先初始化（值为 1），然后 _a2 用 _a1 的值初始化（值为 1）
```

2.类型转换

1. C++支持内置类型隐式转换为类类型对象，需要有相关的内置类型为参数的构造函数
2. 构造函数前面加explicit就不再支持隐式类型转换
3. 类类型的对象之间也可以隐式转换，需要对应的构造函数支持

```
class A {
public:
    //构造函数explicit就不再支持隐式类型转换
    //explicit A(int a1)
    A(int a1)
        :_a1(a1) {

    }
    //explicit A(int a1,int a2)
    A(int a)
        :_a1(a)
        ,a2(_a1) {
    }
    void Print() {
        cout << _a1 << " " << _a2 << endl;
    }
    int Get()const {
        return _a1 + _a2;
    }
private:
    int _a1 = 2;
    int _a2 = 2;
};
class B
{
public:
    B(const A& a)
        :_b(a.Get()) {

    }
private:
    int _b = 0;
};
int main() {
    //构造一个临时对象，再用这个临时对象拷贝aa3
    //编译器遇到连续构造+拷贝构造->优化为直接构造
    A aa(1);
    aa.Print();

    const A& aa2 = 1;
    //C++之后才支持多参数转化
```

```
    A aa3 = { 2,2 };
    B b = aa3;
    const B& rb = aa3;
    return 0;
}
```

3.static成员

1. 用static修饰的成员变量，称之为静态成员变量，静态成员变量一定要在类外进行初始化
2. **静态成员变量为所有类对象所共享**，不属于某个具体的对象，不存在对象中，存放在静态区
3. 用static修饰的成员函数，称之为静态成员函数，**静态成员函数没有this指针**。
4. 静态成员函数中可以访问其他的静态成员，但是不能访问非静态的，因为没有this指针
5. 非静态成员函数，可以访问任意的静态成员变量和静态成员函数
6. 突破类域就可以访问静态成员，可以通过类名::静态成员或者对象静态成员来访问静态成员变量和静态成员函数
7. 静态成员也是类的成员，受pubic、protected、private访问限定符的限制
8. 静态成员变量不能在声明位置给缺省值初始化，因为缺省值是个构造函数初始化列表的，静态成员变量不属于某个对象，不走构造函数初始化列表。

```
class A
{
public:
    A() {
        ++_scount;
    }
    A(const A& t) {
        ++_scount;
    }
    ~A() {
        --_scount;
    }
    static int GetACount() {
        return _scount;
    }
private:
    //类里面申明
    static int _scount;
};

//类外面初始化，它不属于某一个对象，静态成员变量不能在声明位置给缺省值初始化
int A::_scount = 0;
```

```
int main() {
    cout << A::GetACount() << endl;
    A a1, a2;
    A a3(a1);
    cout << A::GetACount() << endl;
    cout <<a1.GetACount() << endl;
    //编译报错："A::_scount":无法访问private成员
    //cout<<A::_count<<endl;
}
```

4.友元

4.1什么是友元？友元的分类是什么？

友元提供了一种突破类限定符封装的方式，友元分为：

1. 友元函数：我们来举一个实际的例子
   代码示例1：

```
//1.友元函数
class Fridge {
private:
    int milk = 10;//假设你家的冰箱里有一瓶牛奶
    //普通朋友来你家是不能喝你的牛奶的，但是你有一个忘年交，你允许它随便喝你的niunai
    //这样你就给你的朋友开一个特权
    friend void takeMilk(Fridge& f);
};

void takeMilk(Fridge& f) {
    f.milk--;
    cout << "你的牛奶我喝了一瓶还剩：" << f.milk << "瓶" << endl;
}
int main() {
    Fridge bestFriend;//这天你最好的朋友来看你的冰箱
    takeMilk(bestFrind);//拿走了你的一瓶牛奶
    //即：外部函数调用，直接操作私有成员
    return 0;
}
```

1. 友元类

```
//2.友元类
//1.友元函数
class Fridge {
private:
    int milk = 10;
    friend class BestFrind;//直接声明整个BestFriend类是友元
};
class BestFrind {
public:
    //BestFrind是Fridge的友元，所以这里能直接访问f.milk
    void stealMilk(Fridge& f) {
        f.milk -= 5;
        cout << "好朋友直接拿走5瓶牛奶" << endl;
    }
    void checkMilk(Fridge& f) {
        cout << "好朋友检查你的牛奶还剩" << f.milk << "瓶" << endl;
    }
};
int main() {
    Fridge bestFrind;//这天你最好的朋友来看你的冰箱
    BestFrind bf;
    bf.stealMilk(bestFrind);
    bf.checkMilk(bestFrind);
    return 0;
}
```

在函数声明或者类声明的前面加friend,并且把友元声明放到一个类的里面。

4.2 友元的作用是什么？怎么使用友元?

1. 外部友元函数可访问类的私有和保护成员，友元仅仅只是一种声明，他不是类的成员函数
2. 友元函数可以在类定义的任何地方声明，不受类访问限定符限制
3. 一个函数可以是多个类的友元函数

```
//实例1：一个函数可以是多个类的友元函数
class Girl;
class Boy {
private:
    int money = 100;//这个男孩有100元
    friend void GoMovice(Boy& boy, Girl& girl);
};
```

```
class Girl {
private:
    int money = 100;//这个女孩有100元
    friend void GoMovice(Boy& boy, Girl& girl);
};
void GoMovice(Boy& boy, Girl& girl) {
    if (boy.money + girl.money>=100) {
        cout << "两个人可以一起去看电影" << endl;
    }
    else
    {
        cout << "钱不够，两个人不可以一起去看电影" << endl;
    }
}
int main() {
    Boy boy;
    Girl girl;
    GoMovice(boy, girl);
    return 0;
}
```

1. 友元类中的成员函数都可以是另一个类的友元函数，都可以访问另一个类中的私有和保护成员

```
class Child {
private:
    int score = 90;
    string secret = "我今天作业没写";
    int money = 100;//孩子有1001元零花钱
    friend class Mother;//声明整个Mother是友元
};

class Mother {
public:
    void checkScore(Child& boy) {
        cout << "这个男孩的成绩为：" << boy.score << endl;
    }
    void checkMoney(Child& boy) {
        cout << "这个男孩的零花钱为：" << boy.money << endl;
    }
    void changeMoney(Child& boy,int x){
        boy.money += x;
        cout << "这个男孩的零花钱为：" << boy.money << endl;
    }
```

```
};
int main() {
    Child child;
    Mother mother;
    mother.checkMoney(child);
    mother.checkMoney(child);
    mother.changeMoney(child, 20);//男孩的妈妈看到男孩考了九十分，奖励了男孩20元
    return 0;
}
```

4.3 友元的特性是什么?

1. 友元类的关系是单向的，不具有交换性，比如A是B的友元，B是C的友元，但是不能说A是C的友元
   示例代码1：

```
class B;
class A {
private:
    int secretA = 100;
public:
    void showBSecret(B& b);//声明成员函数
};
class B {
private:
    int secretB = 200;
    //让A成为B的友元
    friend class A;
public:
    void showASecret(A& a);//声明成员函数
};
//A的成员函数定义(因为是友元，所以能访问B的私有成员）
void A::showBSecret(B& b) {
    cout << "A 访问 B 的秘密：" << b.secretB << endl;
}
//这里编译器会报错a.secretA会显示A类中没有这个成员
// 实际上就是A没有把A当朋友不能被访问
//void B::showASecret(A& a) {
//  cout << "B 访问 A 的秘密：" << a.secretA << endl;
//}

int main() {
    A a;
    B b;
```

```
    a.showBSecret(b);//A可以访问B的私有成员
    return 0;
}
```

1. 友元类关系不能传递，如果A是B的友元，B是C的友元，但A不是C的友元

4.4 友元存在的意义是什么?

1. 有时提供了便利。但是友元会增加耦合度，破化了封装，所以友元不易多用

5. 内部类

1. 如果一个类定义在另一个类的内部，这个内部类就叫做内部类。内部类是一个独立的类，跟定义在全局相比他只是受外部类类域限制和访问限定符的限制，所以外部类定义的对象中不包含内部类。

```
//示例三：内部类
class Car {//外部类
private:
    string brand = "666";
    int price = 3000000;
public:
    //内部类
    class Engine {
    private:
        string model = "v8";
        int horsepower = 500;
    public:
        void shoeInfo() {
            cout << "发动机型号：" << model << endl;
            cout << "马力：" << horsepower << endl;
            //内部类不能直接访问外部类的非静态成员
            //cout << "汽车品牌：" << brand << endl;
        }
        void showCarInfo(Car& c) {
            // 通过外部类对象可以访问公有成员
            cout << "这辆" << c.brand << "的价格是：" << c.price << "元" <<
endl;
        }
    };
    void text() {
        Engine e;// 外部类成员函数中可以创建内部类对象
```

```
        e.shoeInfo();
    }
};
int main() {
    //1.内部类是独立类，可以在外部创建对象
    Car::Engine engine;//需要限定类域
    engine.shoeInfo();
    //2.外部类对象不包含内部类
    Car mycar;
    cout << "Car 对象的大小：" << sizeof(mycar) << " 字节" << endl;
    // 输出只包含 brand 和 price 的大小，不包含 Engine

    //3.内部类访问外部类需要传入外部类对象
    engine.showCarInfo(mycar);
    return 0;
}
```

1. 内部类默认是外部类的友元类
2. 内部类本质上也是一种封装，当A类跟B类紧密关联，A类实现出来主要就是给B类使用，那么可以考虑把A类设计为B类的内部类，如果放到private/protected位置，那么A类就是B类的专属内部类，其他地方都用不了。

6. 匿名对象

1. 用类型（实参）定义出来的对象叫做匿名对象，相比之前我们定义的类型对象名（实参）定义出来的叫有名对象
   匿名对象的定义：
   类名（实参）；
   类名（）；

```
//示例
string();
string("hello");
vector<int>();
Date(2025, 5, 7);
```

1. 匿名对象生命周期只在当前一行，一般临时定义一个对象当前用一下即可，就可以定义匿名对象。

```
class A{
public:
```

```
    A(int a = 0)
        :_a(a)
    {
        cout << "A(int a)" << endl;
    }
    ~A() {
        cout << "~A()" << endl;
    }
private:
    int _a;
};
class solution {
public:
    int something(int n) {
        return n;
    }
};
int main() {
    //A aa1;

    //A aa1();不能这样定义对象，因为编译器无法识别这到底是一个函数声明还是对象定义
    //但是我们可以这样定义匿名对象，匿名对象的特点是不用取名字
    //它的声明周期只有这一行，我们可以看到下一行会自动调用析构函数
    A();
    A(1);
    A aa2(2);
    //匿名对象在这种场景下就很好用，一次性的东西就不用给其取名字了
    solution().something(10);
    return 0;
}
```

7. 对象拷贝时候的编译器优化

1. 现代编译器会为了尽可能的提高程序的效率，在不影响正确性的情况下会尽可能的减少一些传参和传值返回的过程中可以省略的返回。
2. 如何优化C++标准并没有严格规定，各个编译器会根据情况自行处理。当前主流的相对新一点的编译器对于连续一个表达式步骤中的连续拷贝会进行合并优化，有些更新更“激进”的编译器还会进行跨行表达式的合并优化。
3. Linux下可以将下面的代码拷贝到text.cpp文件中，编译时可以关闭构造相关的优化来调式。